Chapter 4

Progressing from Plan to Prototype

IN THIS CHAPTER

» **Understanding your project's requirements**

» **Coding a prototype with prompts**

» **Blending your code with AI-generated code**

» **Tips for generating better code**

Although it's possible to work with an AI coding assistant to generate working code and even entire programs, acceptable results are far from guaranteed. The quality of the results you'll get depends on several factors, including the LLM you use, the input you give the model, your own coding skills, and how clearly you've defined the project's requirements.

Chapter 3 provides an example of using AI to do something most programmers could do faster and better without AI. In this chapter, you start to learn processes and tips for getting consistent and higher-quality results.

Understanding Project Requirements

If you've ever been thrown into working on a new project that you don't fully understand, you can identify with what a coding assistant would feel like all the time — if a coding assistant could feel anything. Although coding assistants have seen a lot of code, the only way they can be helpful

to you is if they have context about the particular requirements of the code you're writing.

Determining the software requirements

When starting to work with a coding assistant, think about what you need to know before working on a project. Specifically, you need to know the following:

- What will the software do?
- Who are you building the software for?
- Who will use the software?
- Where and how will the software do what it does?
- How will the user interact with the software?
- What languages and technologies will you use to build the software?
- What are the goals of the software?
- Are there any legal or regulatory standards that the software must comply with?

In the world of software development, we call the answers to these questions the *software requirements.* You can specify your project's software requirements by using a *software requirements specification (SRS),* which is a document that describes, in detail, what the planned software will do and how it will be expected to perform. For small projects, a full SRS is usually not necessary. However, some sort of documentation of requirements is essential to any project, and figuring out the requirements for yourself is essential to being able to communicate them to another developer or to an AI coding assistant.

Software requirements can be divided into three broad categories: domain requirements, functional requirements, and non-functional requirements.

Domain requirements

Domain requirements are particular to the category, purpose, or industry in which the software will be used. It's possible for a piece of software to be functional and user-friendly without being acceptable for use because it doesn't meet domain requirements. For example, if you develop an on-

line banking app that doesn't meet the legal and regulatory requirements that apply to online banking apps, it doesn't fulfill its requirements.

Functional requirements

Functional requirements define how the software system behaves and are generally defined using specific responses to inputs or conditions. These statements of functional requirements are called use cases or user stories.

A *use case* is a detailed description of a functional requirement. It defines, using natural language, the ways in which a user can interact with a system (such as a piece of software or a website) and how the system will respond.

The details specified in a use case include the following:

- The goal
- Whether the user (called the *actor* in use cases) is human or another system
- Preconditions that must be present for the use case (for example, the user must be logged in)
- The series of steps the system will take
- Alternative steps (for example, what happens when the user isn't logged in)
- What happens after the steps are complete (also known as *postconditions*)

Use cases are no longer common in modern software development due to the popularity of agile software development, in which functional requirements are specified with user stories.

User stories are generally informal one-sentence statements written from the user's point of view. They contain the who, what, and why of an outcome that the users want to accomplish with the system. User stories are often written using the following format:

> *As a [persona], I [want to], [so that].*

For example:

> *As a user, I want to be able to reset my password if I forget it, so I can regain access to my account.*

This format, however, is not required when writing user stories. It's also common, especially during the initial process of documenting functional requirements, to see less structured statements that may eventually be turned into user stories.

Whether you decide to write use cases or user stories, the following are examples of functional requirements:

- The system must allow users to create an account.
- The system must allow users to log in with a username and password.
- The system must allow users to click a forgot password link to reset their password.
- After a user signs up for an account, they'll see a login page where they can enter their username and password to log in.
- When a user successfully signs in, they'll see the newsfeed page.
- The newsfeed page presents the user with a list of the latest posts made by other users.
- At the bottom of the newsfeed page, users can enter text into an input field and click a submit button to create a new post.

You don't need to capture every detail of the functional requirements for the SRS. However, stating who will be using the system, what needs to be built, and why it needs to be built will give you more clarity as you're writing code and will give your AI assistant more context.

Non-functional requirements

Non-functional requirements relate to the quality of the software system, including security, maintainability, reliability, scalability, and reusability. Prioritizing non-functional requirements often involves considering the different assumptions and constraints that apply to the project.

Assumptions are factors that are believed to be true but aren't confirmed. Following are the categories of assumptions:

- **Technical assumptions** relate to technology, such as hardware, operating systems, and infrastructure. For example, a technical assumption may be that the user's computer will have a certain amount of RAM.
- **Operational assumptions** have to do with user or organizational behavior and factors. For example, an operational assumption for this book is that the reader

is a computer programmer (or wants to be one).

- **Business assumptions** focus on the business context in which the project will be used, such as business policies and market conditions that may affect the design or functionality of the software.
- **Environmental assumptions** are external factors that may affect the software, such as cultural, social, or geographical factors or availability of a stable internet connection.

Constraints restrict the options available to developers of a system. Types of constraints include the following:

- **Interface constraints** define the limitations that the software's user interface puts on the project. If you're developing a mobile app, the size of the mobile device screens places limits on the functionality and design of the app.
- **Performance constraints** define specific performance criteria that the software must meet. For example, you might define a percentage of uptime or a maximum time for the website to finish loading.
- **Operating constraints** relate to existing organizational processes, policies, or practices. Examples include whether the company where the software will be deployed allows the use of certain software libraries or the organization's policies regarding the use of AI coding assistants.
- **Life cycle constraints** have to do with maintainability, portability, and the ongoing life of the software after it's built.
- **Economic constraints** involve the budget for building and operating the software. Finances are a major limitation of any project.



TIP Specifying software requirements is an important part of any project, and with the emergence of AI coding assistants, a skill that every software developer should have. Many books have been written about software requirements, but the most essential one is *Software Requirements,* 3rd Edition (Microsoft Press) by Karl Wiegers and Joy Beatty.

Writing an SRS

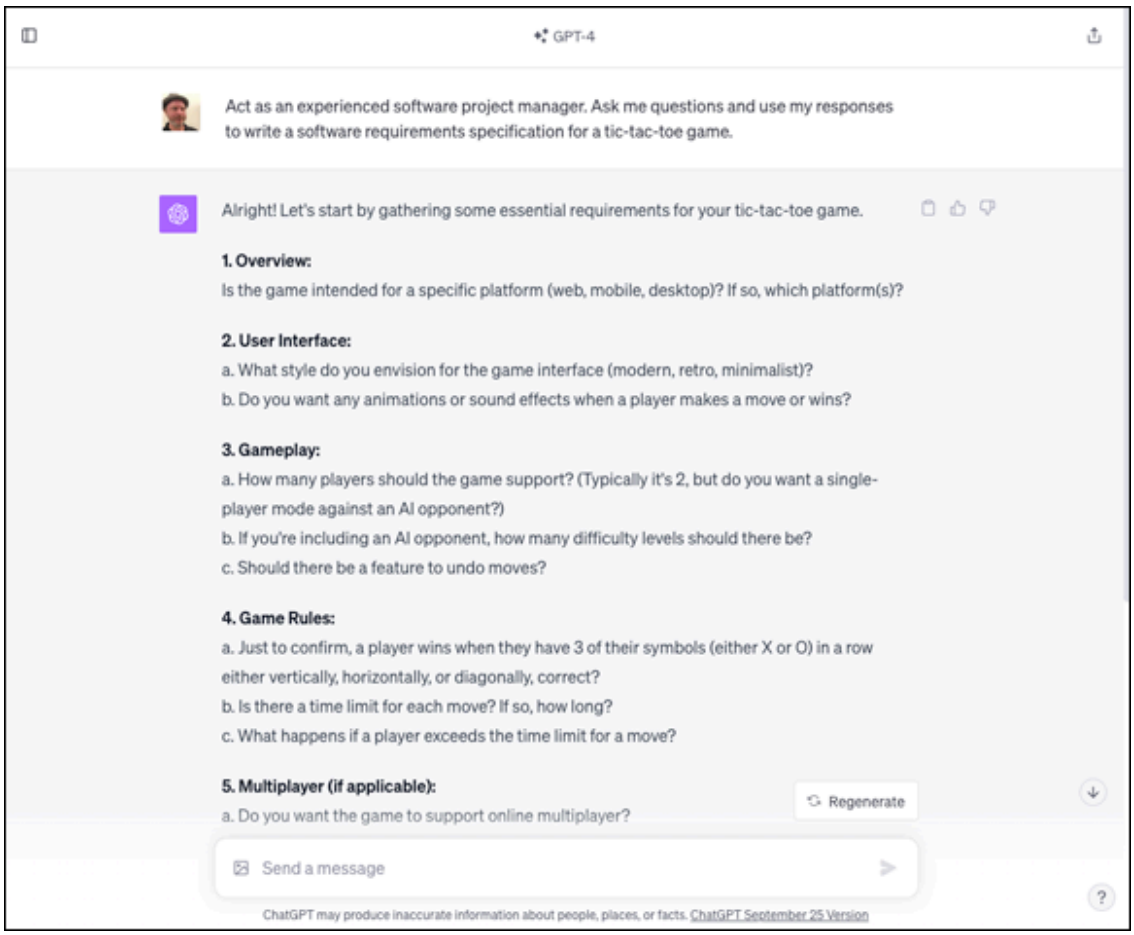
I've decided I want to create a tic-tac-toe game. To help me and my AI coding assistant understand the project, I formulated the following prompt for ChatGPT:

```
Act as an experienced software project manager. Ask me questions and use my responses to write a software requirements specification for a tic-tac-toe game.
```



I'm looking for a more creative response, so I'm using the GPT-4 model rather than the standard (at this time) GPT-3.5 model.

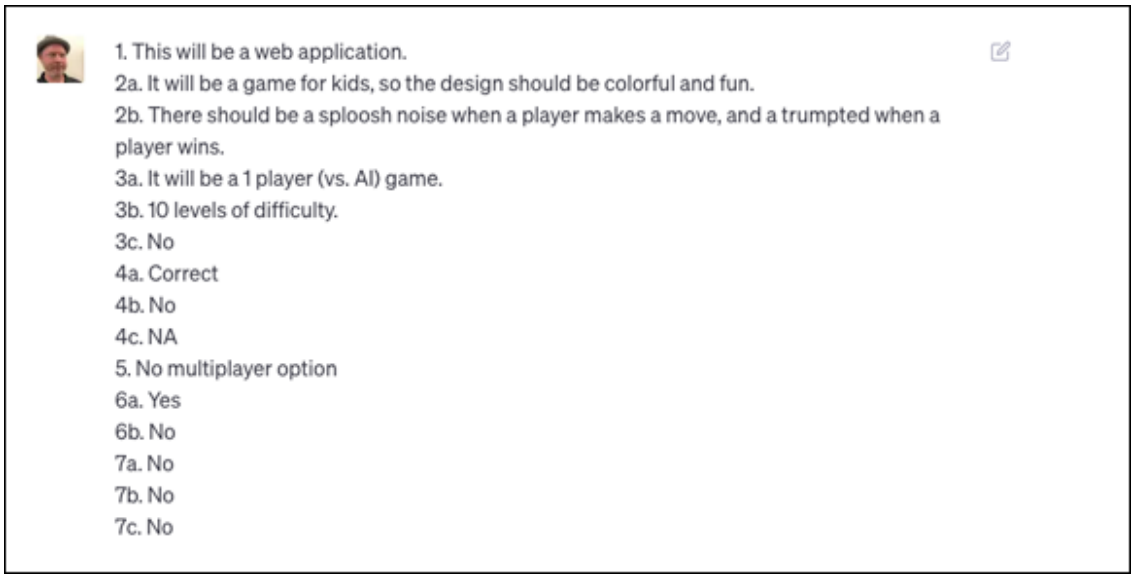
ChatGPT's response is shown in Figure 4-1.



ChatGPT

FIGURE 4-1: Asking ChatGPT to help write an SRS.

My next task is to compose responses to ChatGPT's questions. Because ChatGPT has its questions as context in my current conversation, I prefaced each of my responses with the number and letter of the specific question, as shown in Figure 4-2.



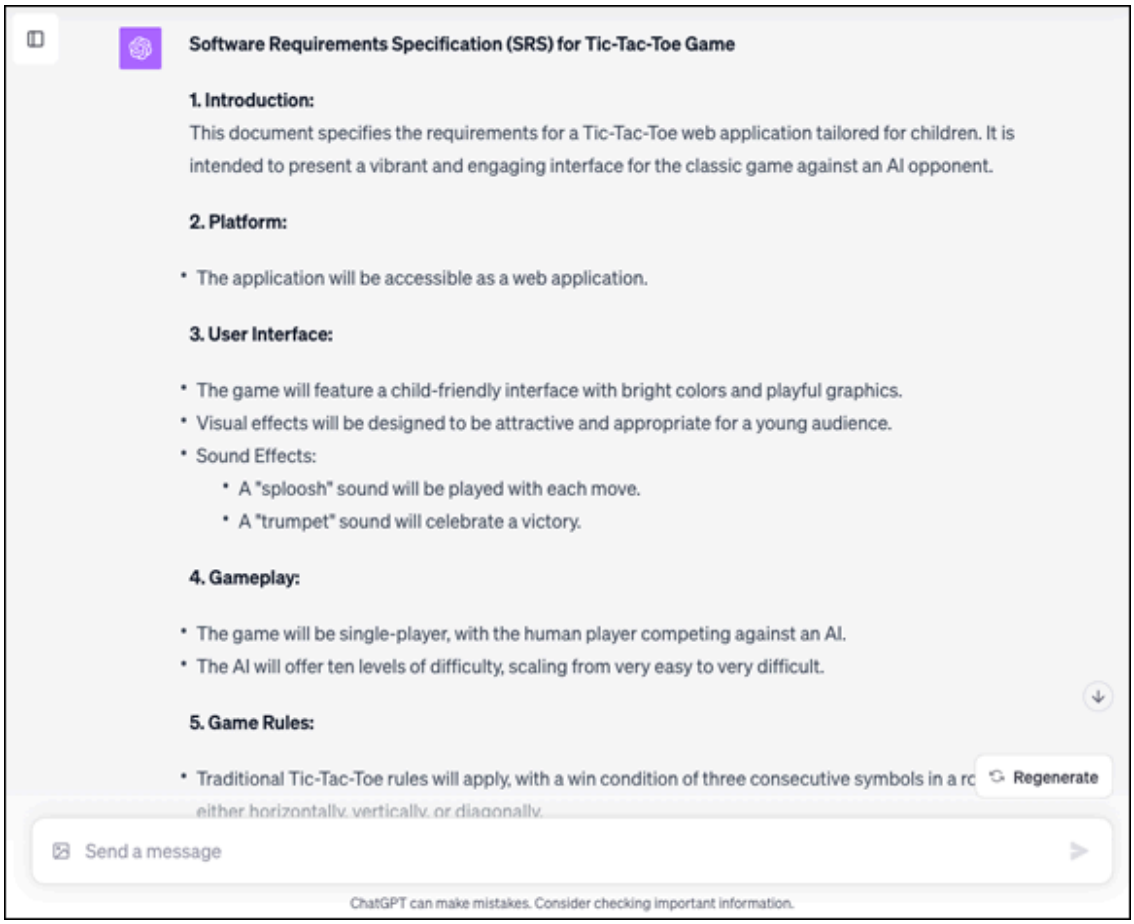
ChatGPT

FIGURE 4-2: My answers to ChatGPT's questions.



TIP To avoid using unnecessary tokens, I entered my response to the questions in a single message by pressing Shift+Enter after each line of my response. This allows ChatGPT to get the entire response at once and avoids making the model respond to each answer individually.

After submitting my answers, ChatGPT responded with the SRS shown in Figure 4-3.

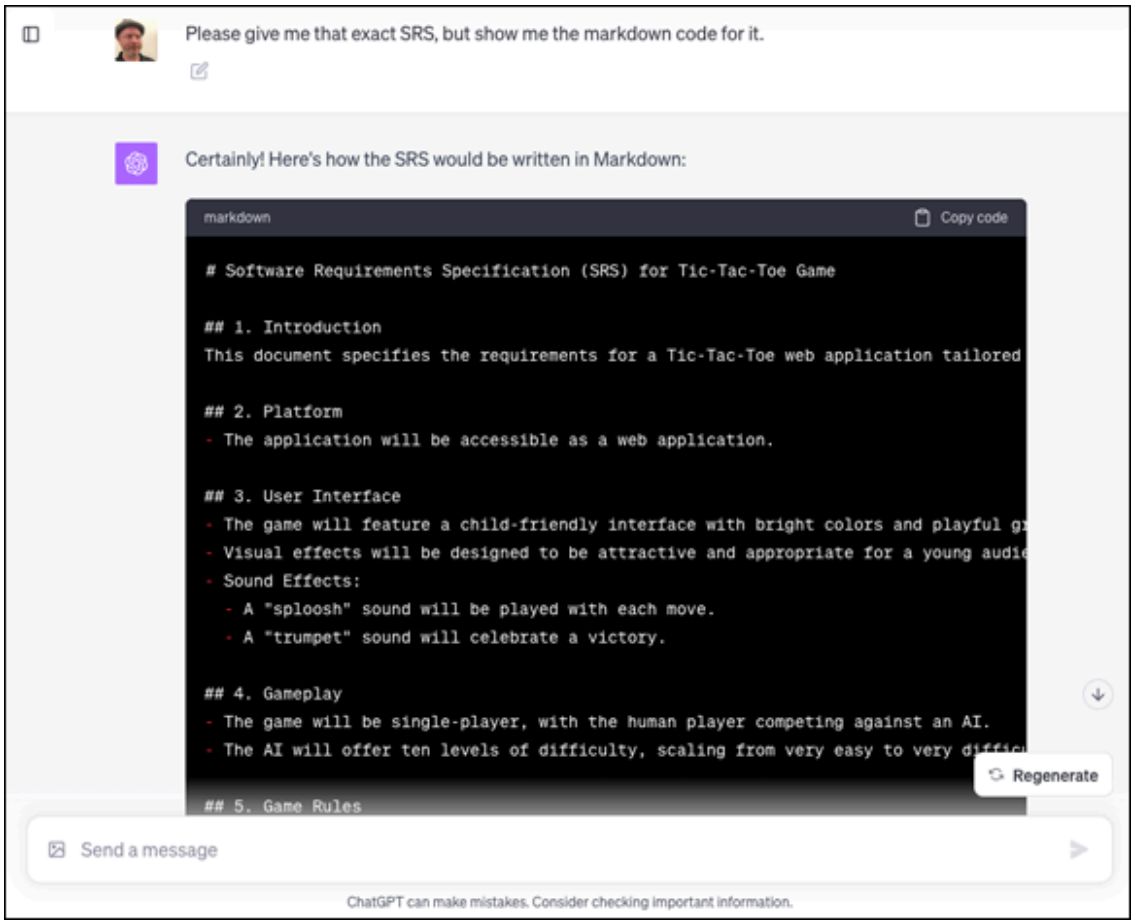


ChatGPT

FIGURE 4-3: A ChatGPT-generated SRS.

The SRS looks good enough, but if I want to keep it in a file outside a ChatGPT window, I'll need it in a more usable format. I prompted ChatGPT to convert the SRS to Markdown syntax. (*Markdown* is a simple markup language for adding formatting to plain-text documents.) The result is shown in Figure 4-4.

I copied the markdown code and saved it in a file with the.md extension in a new Git repository for the project.



ChatGPT

FIGURE 4-4: Converting to Markdown makes documents more usable.

Generating Code from an SRS

Once you have an SRS (whether you wrote it or had your AI assistant's help), you're in a good position to begin coding. You might take a couple of different approaches. The first way is to just ask ChatGPT to code the program described in the SRS. At the very least, this will result in some ideas that you can use to begin coding it yourself.

Using a zero-shot approach

If you're feeling lucky, you can try a simple prompt asking the chatbot to generate the entire program based on the SRS. This approach is called a zero-shot prompt and is described in more detail in Chapter 3.

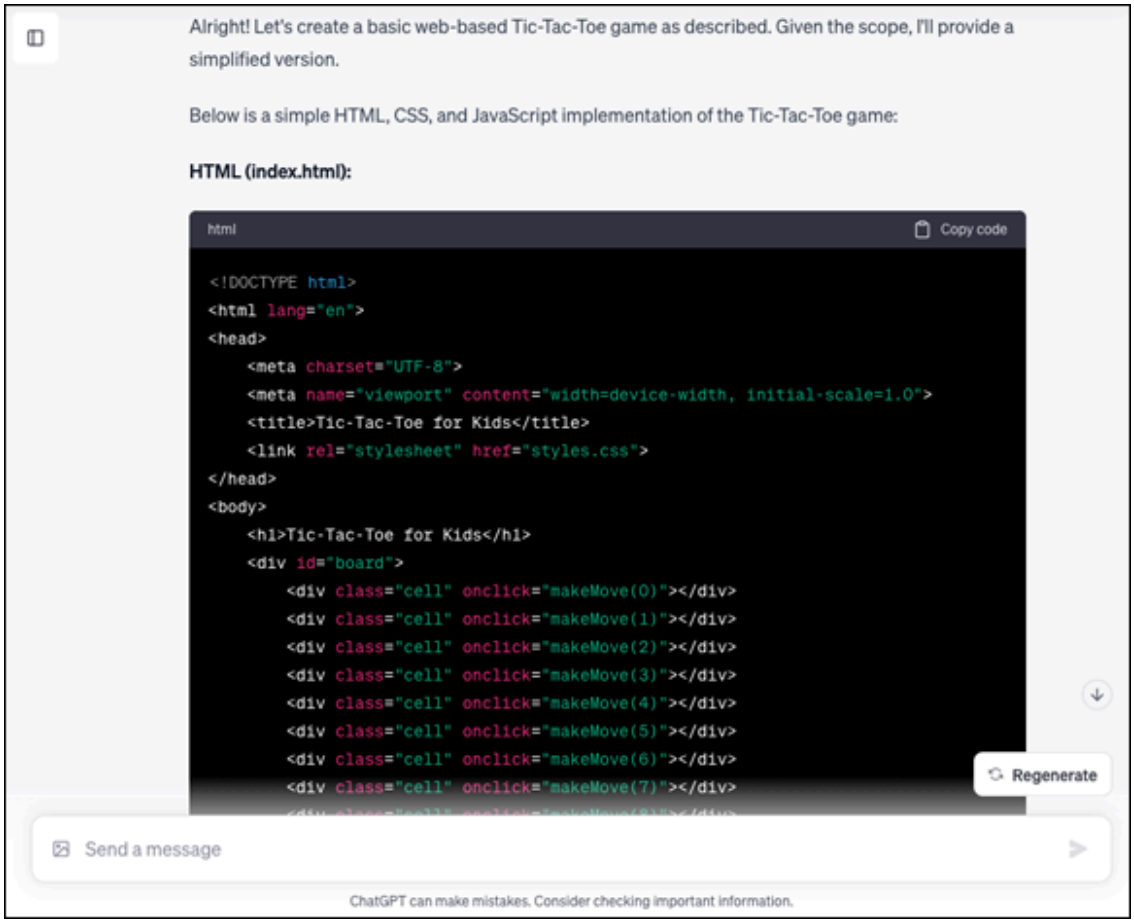


REMEMBER A zero-shot prompt is one in which you don't give examples or code and instead just rely on the data the model has been trained on.

In the same conversation as the one where I asked ChatGPT to write an SRS, I entered the following prompt:

```
You are a professional software developer. Write the tic-tac-toe game described in the above SRS.
```

After a few seconds, ChatGPT returned HTML, CSS, and JavaScript for what it described as a simplified version of the application, as shown in Figure 4-5.



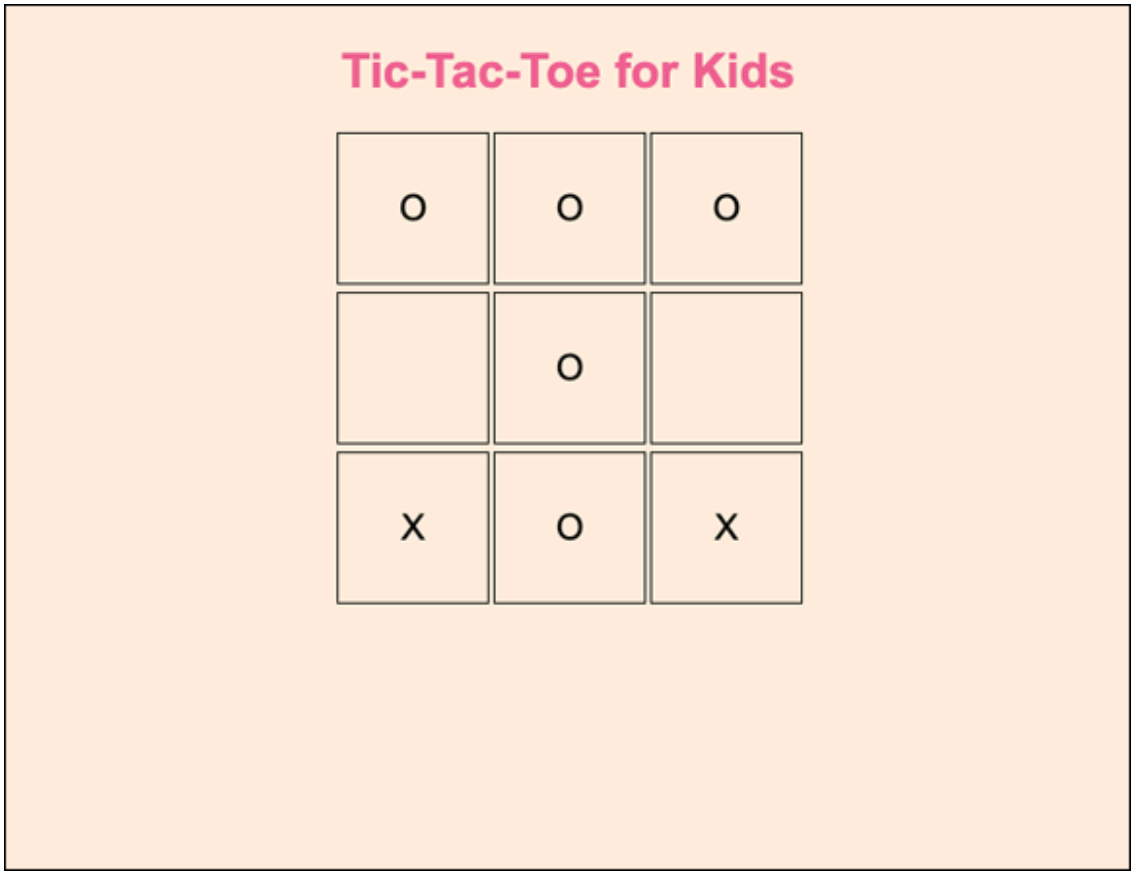
ChatGPT

FIGURE 4-5: The ChatGPT-generated tic-tac-toe game code.



REMEMBER You can find the full source code for this application on the website for *Coding with AI For Dummies* at, `www.dummies.com/go/coding-withaifd`, or in my GitHub repository, at `https://github.com/chrisminnick/aiaio-fd`.

Before examining the code, I copied it into index.html, styles.css, and script.js files in my code editor and opened the HTML file in a browser. The user interface resembles a tic-tac-toe game, and it uses what might be considered kid-friendly colors and text. Upon trying to play the game, however, I discovered that it fails to meet the most critical requirement. Take a look at Figure 4-6 and see whether you can identify the problem here.



ChatGPT

FIGURE 4-6: A frustratingly difficult game of tic-tac-toe.

In this first attempt at a tic-tac-toe game, the AI has written a game that doesn't properly alternate between X and O, resulting in O getting more

turns and, thus, always winning the game. AI cheats.

Breaking down the problem

With a request as complicated as generating software from an SRS, you're much more likely to get acceptable results by breaking down the problem into steps. However, pieces of the output from the zero-shot prompt response can be helpful in figuring out how to break apart the problem.

Since ChatGPT is tuned to be creative (it has a high default temperature), it was the perfect tool for helping with writing an SRS. However, it's not usually the best option for generating working code. For my second attempt at building a working tic-tac-toe game, I turned to GitHub Copilot.

Before asking Copilot to help with my app, I used the built-in debugger in the Chrome browser to figure out what was wrong with the code ChatGPT provided. It turns out that the function that places the user's X on the board has the following line of code to alternate the user's symbol between X and O:

```
currentPlayer = currentPlayer === 'X' ? 'O' : 'X';
```



In this statement, the `currentPlayer` variable holds the symbol (X or O) that will be played on the board when the human user makes a move. The conditional (or ternary) operator in this statement checks whether the value of `currentPlayer` is `X` and changes it to `O` if it is. Otherwise (if `currentPlayer` is not equal to `X`) it changes its value to `X`. The problem, however, is that the value of `currentPlayer` is used in the program only to represent the human player. As a result, switching the value of currentPlayer to `O` makes the human play every other move with the AI's symbol.

After commenting out that line, I was able to play tic-tac-toe and win every time (because the computer player is just randomly picking from the available squares at this point). With a smarter opponent, the game

should be a draw every time, so I decided to implement the levels of difficulty feature from the SRS.

Blending Manually Written and AI-Generated Code

When you have AI-generated code that fulfills at least some of the requirements of the software, it's time to start writing code manually. A good strategy for getting from a basic app, such as the one the AI generated in the preceding section, to something that works correctly is to start by developing the back end.

The *back end* defines the business logic and data in the app, as well as how the user interface will interact with the logic and data. In the case of an AI tic-tac-toe game, writing the back end will start with crafting the prompt that I'll use to get an AI model to play tic-tac-toe with me.

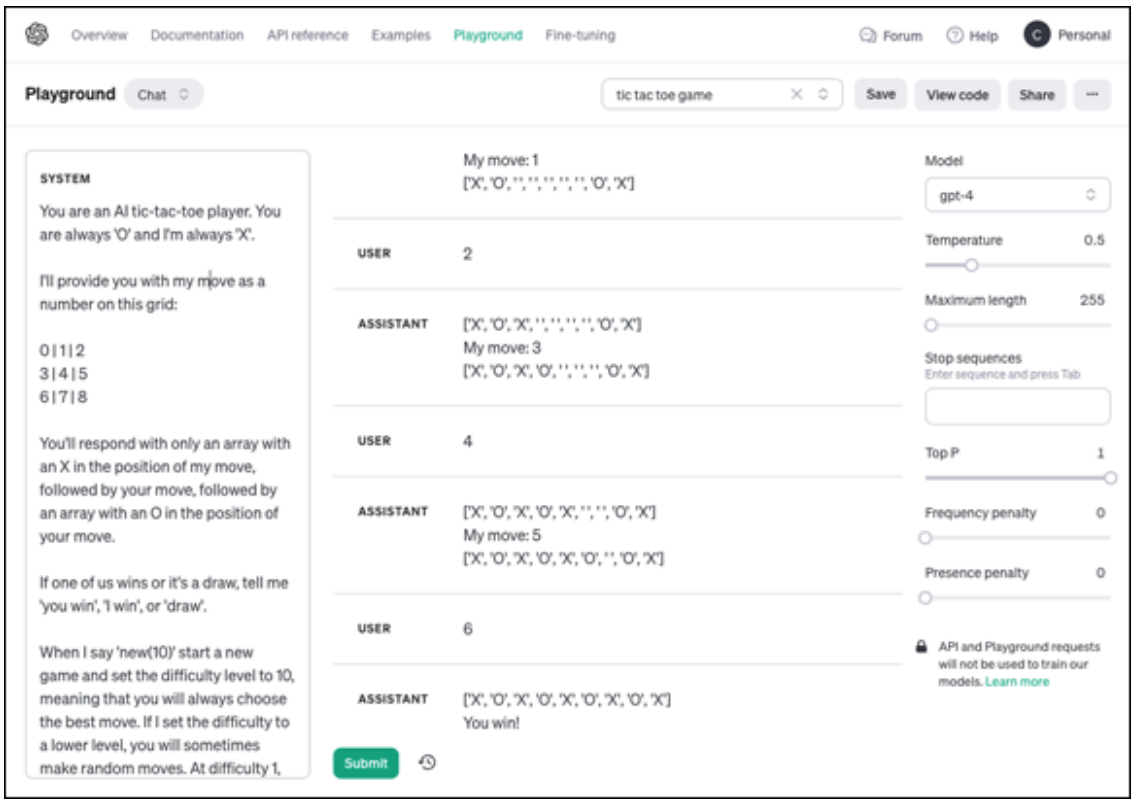
Writing the prompt

After thinking about the problem of how to make an AI tic-tac-toe player, I decided to try using the OpenAI API and the GPT-4 model. I experimented for a while with the OpenAI playground and ended up with the following system prompt:

```
You are an AI tic-tac-toe player. You are always 'O'
and I'm always 'X'.
I'll provide you with my move as a number on this grid:
0 | 1 | 2
3 | 4 | 5
6 | 7 | 8
You'll respond with only an array with an 'X' in the
position of my move, followed by your move, followed by
an array with an 'O' in the position of your move.
If one of us wins or it's a draw, tell me 'you win',
'I win', or 'draw'.
When I say 'new(10)' start a new game and set the
difficulty level to 10, meaning that you will always
```

```
choose the best move. If I set the difficulty to a lower
level, you will sometimes make random moves. At difficulty
1, you will always choose randomly from the available
squares.
```

Using this system prompt and the temperature set to 0.5 (so the model would be creative but not too creative), I was able to enter my moves and get responses that I should be able to use in my program, as shown in Figure 4-7.



ChatGPT

FIGURE 4-7: An AI tic-tac-toe bot in the OpenAI playground.

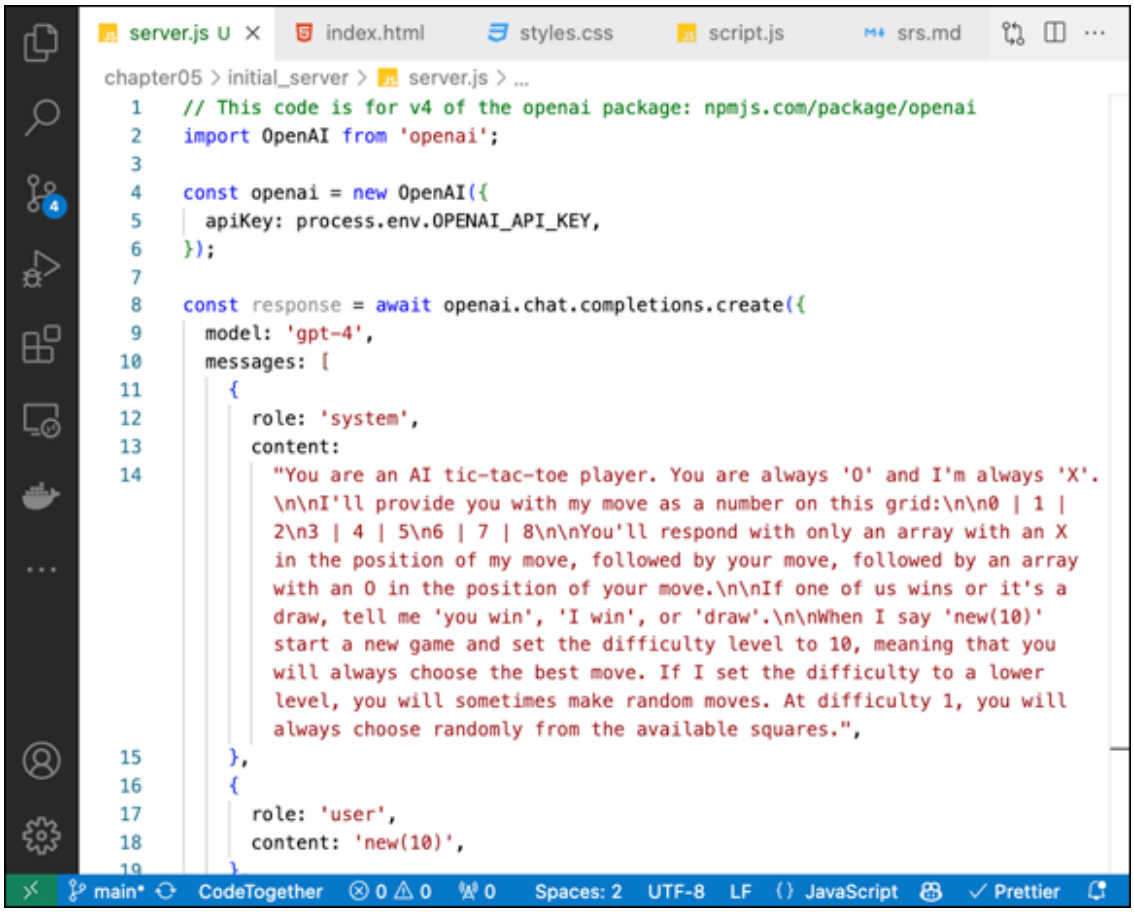
Writing the server

I decided to write a Node.js server to talk to the OpenAI API, so as not to have to store the API key in my client application. I created a new directory for my server, initialized the directory as a Node.js package (using `npm init`), and created a file named server.js.

To start writing server.js, I clicked the View Code button in the upper-right corner of the playground and selected Node.js as my library. The

View Code window opened with the necessary code for sending the current settings and prompts to the API and getting the next completion.

I copied this code from the playground and pasted it into server.js. I also added code to import the API key from a .env file. A .env file is used in Node.js to store environment variables outside the main program. Environment variables hold information that is particular to an installation of the software (such as API keys) and shouldn't be distributed with the software (since you don't want other people using your API key). The beginning of my server is shown in Figure 4-8.



ChatGPT

FIGURE 4-8: Node.js code to get the next completion from the OpenAI API.

The next step was to create my own API server that gets results from OpenAI and returns them to my client application. Since writing an API server is a fairly standard task, I decided to use Github Copilot Chat to generate it.

I started with the following prompt:

```
I want to turn the code in server.js into an API server
that I can send a new message to and have it appended to
the messages array before submitting it to the OpenAI API.
The server should return the response from the OpenAI API.
```

The code generated by Copilot was a good start. The complete code is shown in Listing 4-1.

LISTING 4-1 The First Version of the Tic-Tac-Toe Server

```
import express from 'express';
import OpenAI from 'openai';
import 'dotenv/config';
const app = express();
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});
app.use(express.json());
app.post('/chat', async (req, res) => {
  const { message } = req.body;
  const response = await openai.chat.completions.create({
    model: 'gpt-4',
    messages: [
      {
        role: 'system',
        content:
          "You are an AI tic-tac-toe player. You are al-
ways 'O' and I'm always 'X'.\n\nI'll provide you with my
move as a number on this grid:\n\n0 | 1 | 2\n3 | 4 | 5\n6
| 7 | 8\n\nYou'll respond with only an array with an 'X'
in the position of my move, followed by your move, fol-
lowed by an array with an 'O' in the position of your
move.\n\nIf one of us wins or it's a draw, tell me 'you
win', 'I win', or 'draw'.\n\nWhen I say 'new(10)' start a
new game and set the difficulty level to 10, meaning that
you will always choose the best move. If I set the diffi-
culty to a lower level, you will sometimes make random
```

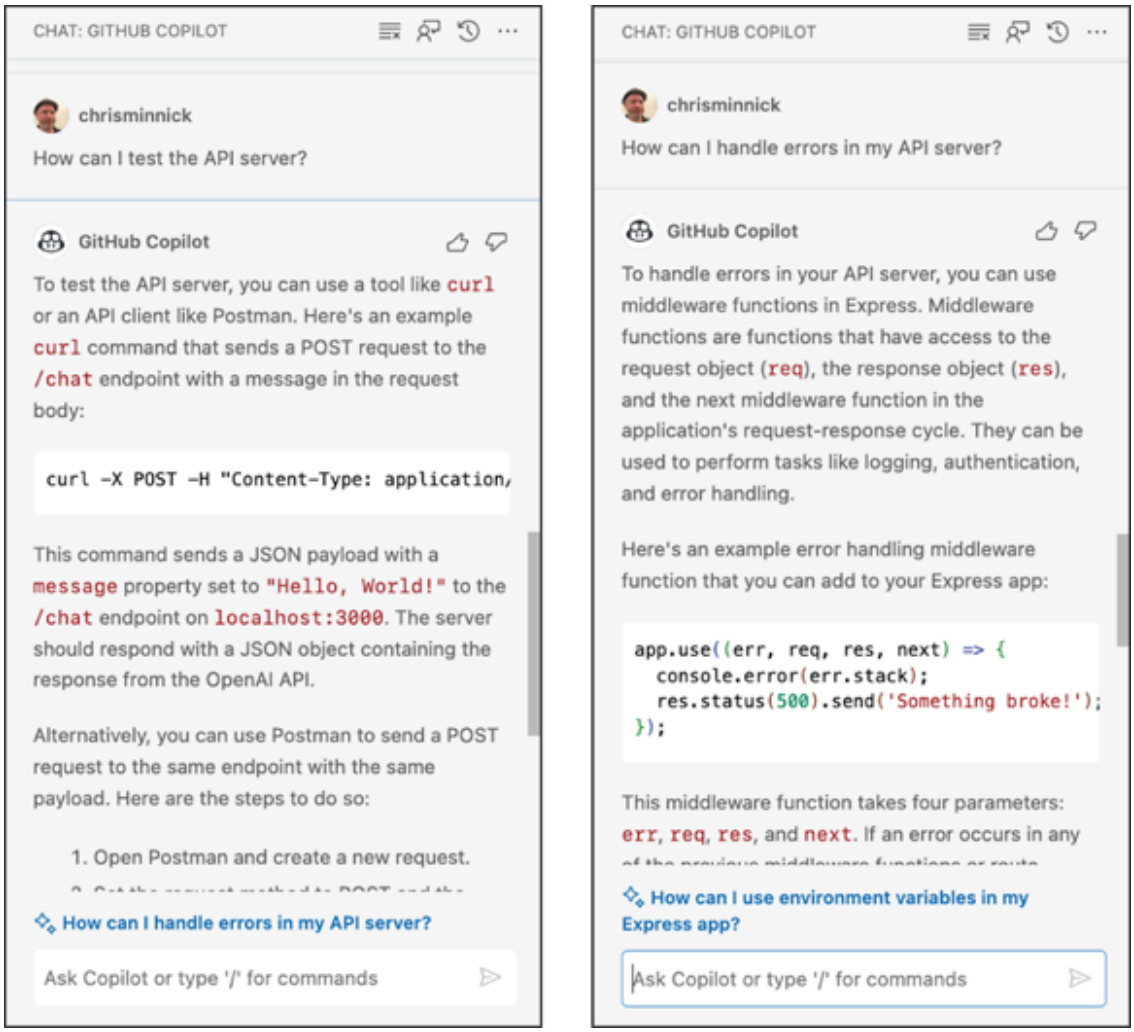
```
moves. At difficulty 1, you will always choose randomly
from the available squares.",
      },
      {
        role: 'user',
        content: message,
      },
    ],
  });
  res.json({ response });
});
app.listen(3000, () => {
  console.log('Server listening on port 3000');
});
```

Submitting follow-up prompts

At the bottom of Copilot's response, it explained the code it generated and suggested a follow-up prompt:

```
How can I test the API server?
```

This was going to be my next prompt anyway, so I clicked the suggested prompt, and Copilot (correctly) suggested using curl or Postman to test it and gave instructions for using both, as shown in Figure 4-9, left.



ChatGPT

FIGURE 4-9: Suggestions for testing the API server (left) and for handling API server errors.

The model also suggested another follow-up prompt, which seemed like an excellent idea to me, so I clicked that to ask for code for handling errors returned by the API server. The response to that prompt is shown in Figure 4-9, right.

Testing the server

Armed with Copilot's suggestions for testing and error handling, I had several tasks to work on before I could make the server work with the client app. First, I had to figure out whether the server works at all. From browsing the code, it looks like a standard Node.js API server that should be able to receive a message, pass it along to the OpenAI server, and return a response in JSON format.

Using the testing suggestions from Copilot, I started the server (using `node server.js`) and entered the following `curl` command into a new terminal window (all on one line):

```
curl -X POST -H "Content-Type: application/json" -d '{"message": "new(5)"}' http://localhost:3000/chat
```

This command should tell the AI tic-tac-toe game to start a new game with a difficulty level of 5. And the response I got from the server shows that was what it did:

```
"message":{"role":"assistant","content":"Understood. We start a new game with a difficulty level of 5, which means a mix of optimal and random moves. Your move!"},"finish_reason":"stop"}],"usage":{"prompt_tokens":199,"completion_tokens":29,"total_tokens":228}}}%
```

Next, I tried submitting my first move, and the AI responded appropriately. However, after a few moves, my AI opponent seemed to lose track of the game and responses were taking far too long. My first attempt at playing tic-tac-toe against the AI using my API server is shown in Figure 4-10.



Microsoft Corporation

FIGURE 4-10: My AI opponent loses track of its instructions.

The problem is that, although this code works, it doesn't have any way to keep track of the state of the game. Each request will only send the latest move from the client to OpenAI. Also, the AI would benefit from some examples of the response format that I'm looking for.



REMEMBER Giving the AI examples of correct responses is called *few-shot prompting*. For details on this type of prompting, see Chapter 3.

Implementing few-shot prompting on the server

To give the AI more context, I wrote a series of messages (using the OpenAI playground) to simulate a correctly played game between the AI assistant and a human user. I then hard-coded those into the server, as shown in Figure 4-11.

```
10  messages: [
11    {
12      role: 'system',
13      content:
14        "You are an AI tic-tac-toe player. You are always 'O' and I'm always 'X'.\n\nI'll
          provide you with my move as a number on this grid:\n\n0 | 1 | 2\n3 | 4 | 5\n6 |
          7 | 8\n\nYou'll respond with only an array with an X in the position of my move,
          followed by your move, followed by an array with an O in the position of your
          move.\n\nIf one of us wins or it's a draw, tell me 'you win', 'I win', or 'draw'.
          \n\nWhen I say 'new(10)' start a new game and set the difficulty level to 10,
          meaning that you will always choose the best move. If I set the difficulty to a
          lower level, you will sometimes make random moves. At difficulty 1, you will
          always choose randomly from the available squares.",
15    },
16    {
17      role: 'user',
18      content: 'new(10)',
19    },
20    {
21      role: 'assistant',
22      content: 'new game',
23    },
24    {
25      role: 'user',
26      content: '0',
27    },
28    {
29      role: 'assistant',
30      content:
31        "['X', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']\nMy move: 4\n['X', ' ', ' ', ' ',
```

FIGURE 4-11: Giving more context to the AI.

I then restarted the server and attempted to play a new game using the `curl` command. The new game started correctly, but on my first move,

the AI responded that I should start a new game to continue playing, as shown in Figure 4-12.



Microsoft Corporation

FIGURE 4-12: The AI doesn't remember the last command.

This happened because the previous prompts in the conversation were not being sent to the server. Since the server has no way of tracking sessions between an individual user and the AI, the best place to implement session state is on the client.

Before working on the client, however, I have to set up the server to combine the prompts that are hard-coded on the server with the prompts that come from the client. I stored the system prompt and the example game in a variable and then prepended that to the messages that the client app sends to the server.

I also simplified the example game so the AI is returning only the number of the square where it wants to place a O. This change makes the client app easier to code and has the additional benefit of greatly reducing the number of tokens necessary for playing a game.

The finished server app is shown in Listing 4-2.

LISTING 4-2 My Finished API Server

```
import express from 'express';
import OpenAI from 'openai';
import 'dotenv/config';
import cors from 'cors';
const app = express();
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});
app.use(express.json());
app.use(cors());
app.post('/chat', async (req, res) => {
  const context = [
    {
      role: 'system',
      content:
        "You are an AI tic-tac-toe player. You are always
'O' and I'm always 'X'.\n\nI'll provide you with my move
as a number on this grid:\n\n0 | 1 | 2\n3 | 4 | 5\n6 | 7 |
8\n\nYou'll respond with only your move, which must not be
a number that has already been played in the current
game.\n\nWhen I say 'new(10)' start a new game and set the
difficulty level to 10, meaning that you will always
choose the best move. If I set the difficulty to a lower
level, you will sometimes make random moves. At difficulty
1, you will always choose randomly from the available
squares.",
    },
    {
      role: 'user',
      content: 'new(10)',
    },
    {
      role: 'assistant',
      content: 'new game, level 10',
    },
```

```
    {
      role: 'user',
      content: '0',
    },
    {
      role: 'assistant',
      content: '4',
    },
    {
      role: 'user',
      content: '1',
    },
    {
      role: 'assistant',
      content: '2',
    },
    {
      role: 'user',
      content: '6',
    },
    {
      role: 'assistant',
      content: '8',
    },
    {
      role: 'user',
      content: '5',
    },
    {
      role: 'assistant',
      content: '3',
    },
    {
      role: 'user',
      content: '7',
    },
  ];
```

```
  const newMessage = req.body.messages;
  const messages = [...context, ...newMessage];
  const response = await openai.chat.completions.create({
    model: 'gpt-4',
    messages: messages,
    temperature: 0.5,
    max_tokens: 255,
    top_p: 1,
    frequency_penalty: 0,
    presence_penalty: 0,
  });
  res.json({ response });
});
app.listen(3000, () => {
  console.log('Server listening on port 3000');
});
```

Improving the client

To make the client application send my moves to the server and get the AI's moves back from the server, I wrote a new function named `getAIMove()`. This function sends the API server all the moves in the current game and gets back the AI's new move.

I also created a function named `startNewGame()` that takes the level of difficulty and passes the command to the server to start a new game. The complete code for the client-side script is shown in Listing 4-3.

LISTING 4-3 The Client-Side JavaScript

```
let board = ['', '', '', '', '', '', '', '', ''];
let currentPlayer = 'X';
let isGameOver = false;
let messageHistory = [];
function startNewGame(levelOfDifficulty) {
  board = ['', '', '', '', '', '', '', '', ''];
  currentPlayer = 'X';
```

```
  isGameOver = false;
  messageHistory = [];
  messageHistory.push({
    role: 'user',
    content: 'new(' + levelOfDifficulty + ')',
  });
  document.querySelectorAll('.cell').forEach((cell) =>
(cell.innerHTML = ''));
  const response = getAIMove(messageHistory);
  return response;
}
function makeMove(index) {
  if (board[index] === '' && !isGameOver) {
    board[index] = currentPlayer;
   document.getElementsByClassName('cell')[index].inner-
HTML = currentPlayer;
    messageHistory.push({
      role: 'user',
      content: index.toString(),
    });
    if (checkWin()) {
      alert(currentPlayer + ' Wins!');
      isGameOver = true;
      return;
    }
    if (checkDraw()) {
      alert('Draw!');
      isGameOver = true;
      return;
    }
    aiMove(messageHistory); // Player is X, AI is O
  }
}
async function getAIMove(message) {
  // This function will send a message to the API server
  // The message will contain each previous move and the
user's latest move
```

```
  // The API server will return the AI's next move
  const response = await
fetch('http://localhost:3000/chat', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      messages: message,
    }),
  });
  const data = await response.json();
  document.getElementById('message').innerHTML =
    data.response.choices[0].message.content;
  return data.response.choices[0].message.content;
}
async function aiMove(messageHistory) {
  let move = await getAIMove(messageHistory);
  messageHistory.push({
    role: 'assistant',
    content: move.toString(),
  });
  board[move] = 'O';
  document.getElementsByClassName('cell')[move].innerHTML
= 'O';
  if (checkWin()) {
    alert('O Wins!');
    isGameOver = true;
  }
}
function checkWin() {
  let winCombos = [
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
    [0, 3, 6],
    [1, 4, 7],
```

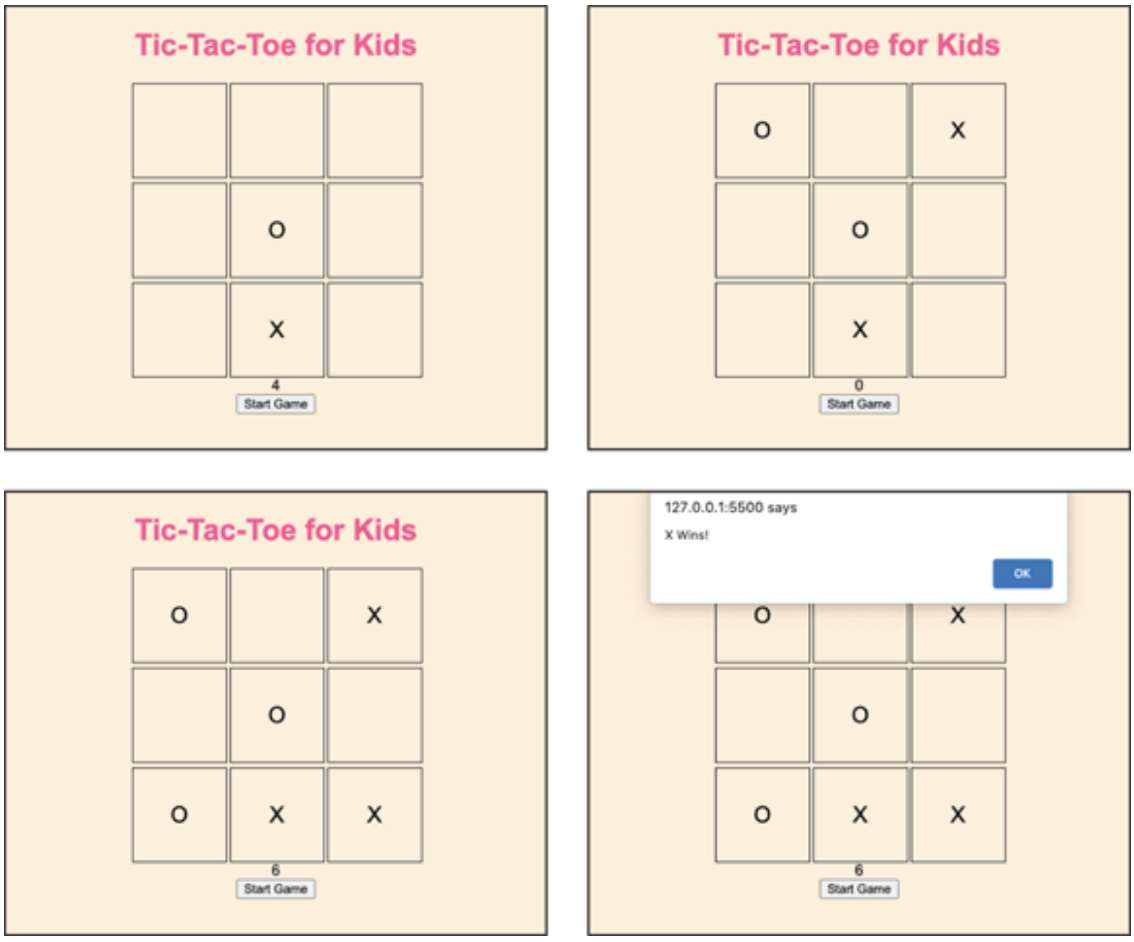
```
    [2, 5, 8],
    [0, 4, 8],
    [2, 4, 6],
  ];
  for (let i = 0; i < winCombos.length; i++) {
    if (
      board[winCombos[i][0]] &&;
      board[winCombos[i][0]] === board[winCombos[i][1]]
&&;
      board[winCombos[i][0]] === board[winCombos[i][2]]
    ) {
      return true;
    }
  }
  return false;
}
function checkDraw() {
  return board.every((cell) => cell !== '');
}
```

Once I finished writing the client-side script and updating the HTML page to add the Start Game button, I tested the game. After a little debugging, the game worked, and I could play tic-tac-toe with GPT-4 through my web browser.

However, after several games, it became apparent that although GPT-4 knows the rules of tic-tac-toe, it is terrible at strategy. I won every game, even when I set the level of difficulty to 10 and I played wrong, as shown in Figure 4-13.



REMEMBER GPT-4 model is a language model, and it's not well-equipped to handle reasoning.



ChatGPT

FIGURE 4-13: GPT-4 is no good at tic-tac-toe.

Moving logic from AI to the client

When integrating responses from an AI into an app, consider whether parts of the response from the AI can be done in your client- or server-side code. If so, you can reduce the complexity of the instructions to the AI as well as the number of interactions between your application and the AI. This will have multiple benefits, including reduced costs for AI usage, improved performance, and allowing the AI to focus on fewer tasks, which may improve its accuracy.

Because the level of difficulty simply adjusts the number of random responses, making random moves seems like a natural thing to take off the AI's plate. My idea was to handle the level of difficulty on the client side, and not even prompt the AI for random moves.

To accomplish this, I went back to the completely random tic-tac-toe move code that ChatGPT generated and modified it so that the selected level of

difficulty would determine how frequently moves are generated randomly.

First, I revised the system prompt to eliminate the description of the levels of difficulty. Here's my new prompt:

```
{
      role: 'system',
      content:
        "You are an AI tic-tac-toe player. You are always
        'O' and I'm always 'X'.\n\nI'll provide you with
        my move as a number on this grid:\n\n0 | 1 | 2\n3
        | 4 | 5\n6 | 7 | 8\n\nYou'll respond with only
        your move, which must not be a number that has
        already been played in the current game.\n\nWhen I
say 'new()' start a new game.",
}
```

Then, to start writing the random move functionality, I entered the following comment at the beginning of the `getAIMove()` function:

```
/*
  Use the value of difficulty to decide whether to
  query the API for a move or use a random move.
  If difficulty is 10, always query the API
  If difficulty is 0, always use a random move
  If difficulty is between 0 and 10, use a random move
  (100 - difficulty x 10) percent of the time
  and use the best move (difficulty x 10) percent of the
time
*/
```

The modified function I wrote with the help of Copilot is shown in Listing 4-4.

LISTING 4-4 The new getAIMove() Function

```
async function getAIMove(message) {
  /*
  Use the value of difficulty to decide whether to
  query the API for a move or use a random move.
  If difficulty is 10, always query the API
  If difficulty is 0, always use a random move
  If difficulty is between 0 and 10, use a random move
  (100 - difficulty x 10) percent of the time
  and use the best move (difficulty x 10) percent of the
time
*/
  let random = Math.random();
  if (random < difficulty / 10) {
    const response = await
fetch('http://localhost:3000/chat', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        messages: message,
      }),
    });
    const data = await response.json();
    document.getElementById('message').innerHTML =
      data.response.choices[0].message.content;
    return data.response.choices[0].message.content;
  }
  let move = Math.floor(Math.random() * 8);
  while (board[move] !== '') {
    move = Math.floor(Math.random() * 8);
  }
  document.getElementById('message').innerHTML = move.to-
String();
  return move.toString();
}
```

With this new prompt and function, I could set the difficulty (in the script at first, and then through the use of a slider input in the HTML) and adjust the percentage of moves made by GPT-4 versus moves randomly generated in the client application.

The AI still couldn't play tic-tac-toe worth a darn, however, even with the difficulty level set to 10.



TECHNICAL STUFF It may be possible, though better prompting, to get GPT-4 to be good at tic-tac-toe. Email me at `chris@minnick.com` if you figure it out!



REMEMBER You can find the complete code for the tic-tac-toe game on this book's website at `www.dummies.com/go/codingwithaifd`.

Tips and Tricks for Code Generation

The results you get from a generative AI model will vary widely, based on your prompts, the context and input you provide to the model, the specific LLM you use, the temperature and other settings provided to the model, and more.

As you become more comfortable working with an AI coding assistant, you'll start to become more familiar with what it can and can't do. If you follow certain practices, however, you can get the LLM to generate good code more reliably. In this section, I tell you some of the tips and tricks that I've found to be most helpful, as well as a few practices that will end up costing you more time than it's worth to use the AI.

Don't stop coding

While AI can generate complete functions, or even working programs, it works best when you take the lead. Keep your skills up-to-date and use AI as a tool to help you write more code, rather than as a tool that will write

your code for you. Not only will the AI learn from you and write better code, but you'll also be able to fully understand the code you're writing, which is essential to creating high-quality software.

Be specific

When prompting a coding assistant to generate code, or when asking a question about how to do something, be as specific as possible. When you're exploring an idea that you don't yet know how to code, it can be helpful to start with a vague prompt to generate ideas. But after you understand the problem and the domain, ask for details.

Think in steps

Complex requests are much more likely to result in unacceptable responses. Instead, break down every problem into its smallest units. Instead of saying, "How do I write an Instagram clone?" start with a plan (which you should consider writing as an SRS) and then with a small piece, such as the new user signup page.

Ask follow-up questions

If you're unsure about how a piece of generated code works or whether a response is unexpected, ask the AI assistant to clarify, explain, or try again. For example, if a function generated by the AI doesn't look quite right to you, but you don't know exactly what's wrong with it, ask the AI to provide several other ways that the function might be written. Just as you might watch several YouTube videos before finding the best solution to a problem, having your AI assistant generate options can be a great way to figure out what will work best. If you don't like the responses one AI assistant gives you, try another AI assistant. Or try feeding the code generated by one AI assistant to another one, asking it to improve the code.

Check the official documentation

Remember that coding assistants are trained on publicly available code, and the LLM behind the assistant may have a training data cutoff date. As a result, code generated by an LLM may use deprecated syntax, an older version of a library, or libraries that are no longer recommended. If a code assistant uses syntax or a library that you're not familiar with, check the official documentation to make sure you're using it correctly.

Use examples and context

Although the latest LLMs have amazing capabilities, they aren't mind readers. If you want the output from a request to be in a specific format, give the AI an example of that format. If the code you ask the AI to generate will integrate with some other function or service, provide the relevant information to the AI as context.

Prioritize security

Chatbots and coding assistants may use your input to train the underlying model. To be certain that no sensitive or personally identifying information will show up in suggestions given to other users of the LLM, always anonymize sensitive data. For example, if you ask the AI to summarize a long email, remove or change the email addresses and names from the email before submitting it as input to an LLM.

Keep learning

Working with a chatbot or a coding assistant is a great way to get answers to coding questions, but it's no substitute for staying up-to-date the same way programmers always have — by talking with other programmers, engaging in online forums on StackOverflow, Reddit, and Hacker News, watching videos, taking classes, and reading high-quality books like the one you have in front of you!

Keep your tools updated

AI coding tools are evolving quickly. Make sure that you have the latest version of whatever tools you're using. If you learn about a new tool that

seems promising (through any of the sources mentioned in the preceding tip), try it out. If a new tool or IDE works better for you than the one you've been using, consider switching to it.

Be mindful of AI’s limitations

As you see repeatedly in this book, LLMs aren’t perfect or omniscient. They were trained on a lot of data and can make predictions based on patterns they find. If you’re writing code that’s unlike anything the AI has seen before, it will be of little help to you.